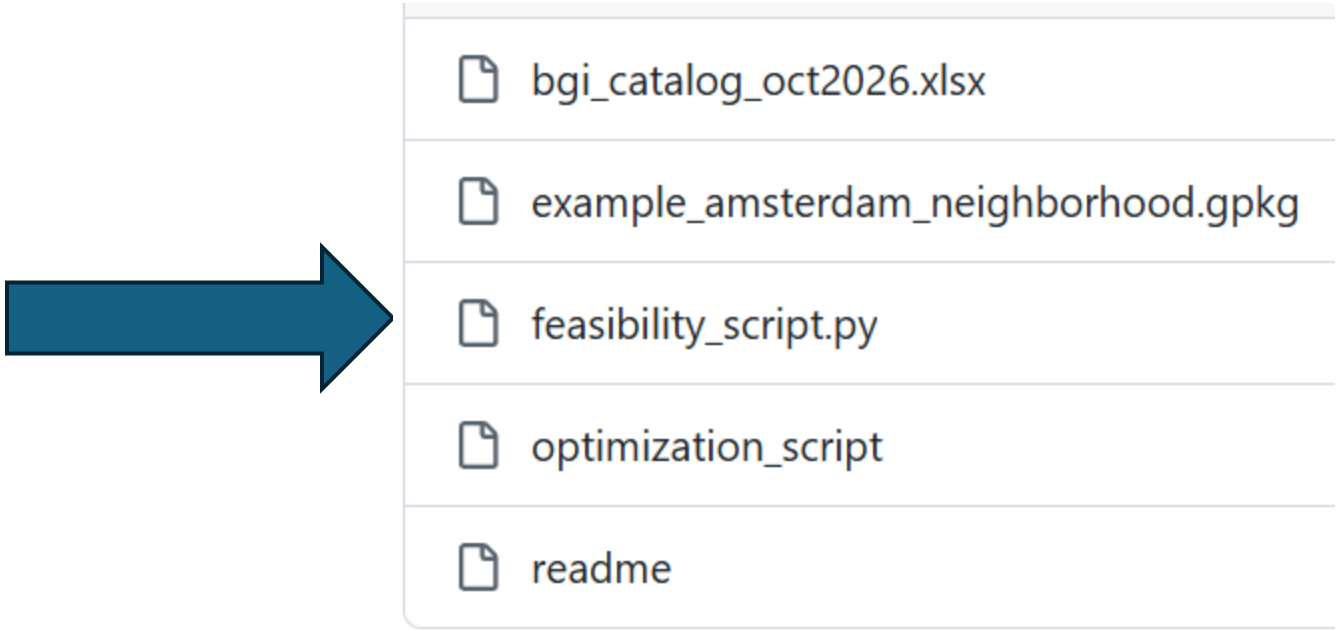
BGIS Feasibility Tutorial

bgishub/BGIS: ongoing project for a decision making support tool for placing blue green solutions



Packages to import

```
################################################################################
# packages to import
################################################################################
import geopandas as gpd
import pyogrio
import pandas as pd
import ast
import numpy as np
import json
```

Necessary Python packages for running script.
JSON and AST are included with Python's standard library

You may need to install packages [GEOPANDAS, PYOGRIO, PANDAS, NUMPY] yourself. Please look into installations on your own as they will be specific to your Python set up.

Pyogrio can be exchanged for Fiona as needed

Functions for reading – no actions needed

```
##############################################################################
# functions for loading text correctly
##############################################################################

def safe_parse(cell):
  """ correctly read excel sheet """
    if pd.isna(cell):                                       # case: missing value
        return None

    if isinstance(cell, str):                               # case: is string
        cleaned_string = cell.strip()

        if cleaned_string.lower() in ["true", "false"]:     # case: Boolean
            return cleaned_string.lower() == "true"

        try:
            parsed = json.loads(cleaned_string)             # attempt JSON parsing
        except Exception:
            parsed = None

        if parsed is None:
            try:
                parsed = ast.literal_eval(cleaned_string)   # attempt literal parsing
            except Exception:
                parsed = cleaned_string
        cell = parsed

    if isinstance(cell, str):                               # case: string
        return cell.strip()

    if isinstance(cell, dict):                              # case: dictionary
        return tuple(sorted(cell.items()))

    return cell


def check_list(value):
  """ for when strings need to be in list """
    if value is None:
        return []
    if isinstance(value, list):
        return value
    return [value]
```

Loading in shapefile

```
##########################################################################
# load shapefile here
# example given is a small section of Amsterdam centrum (a few blocks from Keizersgracht - Prinsengracht)
# shapefile with spatial information in polygons along with a specified land use/landcover deisgnation
# taken from Amsterdam Gemeente data
##########################################################################

LULC_GDF = gpd.read_file('example_amsterdam_neighborhood.gpkg')

# run through safe parsing
LULC_GDF['type'] = LULC_GDF['type'].apply(safe_parse)

# create copy and work with copy from here out
gdf = LULC_GDF.copy()
```

Change to your **file name** here

```
# calculate area if it is not already stored
if "area_m2" in gdf.columns:
    print()
else:
    gdf['area_m2'] = gdf.geometry.area
```

Calculates area of polygons and saves as "area_m2" if the column does not already exist

Landcover categories to "Installation Layer"

Landcover categories are existing types of landcover specific to your shapefile

"Installation Layer" is the simplified version of LULC that is used in matching LULC to BGIs

```
##########################################################################################
# ! important ! dictionary ! here !
# to go from data-specific LULC into corresponding installation layer for BGI
# may be customized to different LULCs
##########################################################################################

landcover_dictionary = {
    'baan_voor_vliegverkeer': 'pavement',
    'bassin': 'feature',
    'berm': 'ground',
    'erf': 'ground',
    'fietspad': 'pavement',
    'flat roof': 'flatroof',
    'gesloten_verharding': 'ground',
    'grasland_overig': 'ground',
    'groenvoorziening': 'ground',
    'half_verhard': 'ground',
    'inrit': 'none',
    'kademuur_V': 'quaywall',
    'lage_trafo': 'infrastructure',
    'muur_V': 'wall',
    'oever_slootkant': 'bank',
    'onverhard': 'ground',
    'open_loods': 'aviary',
    'open_verharding': 'ground',
    'ov-baan': 'pavement',
    'pand': 'angledroof',
    'parkeervlak': 'pavement',
    'perron': 'none',
    'rijbaan_lokale_weg': 'pavement',
    'rijbaan_regionale_weg': 'pavement',
    'sluis': 'infrastructure',
    'struiken': 'ground',
    'verkeerseiland': 'ground',
    'voetgangersgebied': 'pavement',
    'voetpad': 'pavement',
    'voetpad_op_trap': 'stairs',
    'waterloop': 'water',
    'woonerf': 'ground',
    'zand': 'ground'
}
```

Installation Layer is used in the BGI catalog



- bgi_catalog_oct2026.xlsx
- example_amsterdam_neighborhood.gpkg
- feasibility_script.py
- optimization_script

	A	B	C	D	E	F	G	I	J
1	bgi_name	install_layer	install_ty	area_min	area_max	max slope	CAPEX	water_storage_volume	temperature_reduction
2	infiltration_crate	["pavement"]	["unit"]	1.4	null	null	300	0.8	0
3	permeable_paveme	["pavement"]	["area"]	2	null	5	150	0.1	6.6
4	rain_garden	["ground"]	["area"]	5	50	null	55	0.4	1.6
5	blue_roof	["flatroof"]	["area"]	5	null	null	50	0.1	0.5
6	green_roof	["flatroof"]	["area"]	5	null	null	160	0.03	2
7	tree	["ground"]	["unit"]	25	1000	null	180	0.01	8
8	urban_park	["ground"]	["area"]	1000	4000	null	110	0.35	4.3
9	rain_storage	["building"]	["unit"]	1	null	null	160	0.6	0

Creating your custom dictionary

- The switch from LULC to Installation Layer is to simplify the categories of LULC from the data we use for Amsterdam (Basisregistratie grootschalige topografie (BGT) - Data Amsterdam) which contains more details than we need

```
'gesloten_verharding':          'ground',
'grasland_overig':              'ground',
'groenvoorziening':             'ground',
'half_verhard':                 'ground',
```

- In our example, a rain garden can be installed on ground. "Ground" is the umbrella term we give different LULC categories including "closed pavement", "other grassland", "landscaped green", "semi-closed pavement". This assumes any of these LULCs can be turned into a rain garden if desired.

Creating your custom dictionary

- So consider how the LULC categories constrain what kind of BGIs can be installed upon them

This is where the swap occurs

```
# translation occurs here, adding another column to gdf
gdf['install_layer'] = gdf['type'].str.lower().map(landcover_dictionary).fillna('none')
```

Load in the excel file with BGI information

If you have your own BGI catalog, replace the file name here

"sheet_name = 0" refers to the first sheet of the excel file in the case that there are multiple sheets

```
##############################################################################
# load in excel sheet with information on BGI technology here
# excel sheet represents types of BGIs considered as well as any necessary requireme
# can add new categories or modify existing easily
##############################################################################
BGI_SHEET = pd.read_excel('bgi_catalog_oct2026.xlsx', sheet_name=0)

# convert from excel data to a useful python dictionary
BGI_CATALOG = {}

# these need to be loaded as 'strings' and not as 's','t','r',...
load_as_string_headers = [
    "install_layer",
    "geometry",
    "soil_type",
    "adaptive_siting",
    "vegetation",
    "modular",
    "maintenance_requirement",
    "permit"
]

for index, row in BGI_SHEET.iterrows():
    bgi_name = row["bgi_name"]
    entry = {}
    for column in BGI_SHEET.columns:
        if column in load_as_string_headers:
            clean = safe_parse(row.get(column))
            entry[column] = clean
        else:
            entry[column] = row.get(column)
    BGI_CATALOG[bgi_name] = entry
```

Defining binary checks of conditions for:

- installation layer
- minimum area
- soil category
- groundwater level

```
################################################################################
# creating masks for various condition checks
################################################################################
```

```
def check_install_layer(gdf, catalog):
  """ check if the installation layer of the land matches one of the BGIs """
    catalog_layers = {
        name: set(entry["install_layer"])
        for name, entry in catalog.items()
    }
    series = gdf["install_layer"]
    return pd.DataFrame({
        name: series.isin(layers)
        for name, layers in catalog_layers.items()
    })
```

```
def check_area(gdf, catalog):
  """ check if the land polygon has the minimum or maximum area as specified by BGI characteristic """
    catalog_min_area = {
        name: float(entry["area_min"]) if pd.notna(entry["area_min"]) else 0
        for name, entry in catalog.items()
    }
    catalog_max_area = {
        name: float(entry["area_max"]) if pd.notna(entry["area_max"]) else np.inf
        for name, entry in catalog.items()
    }
    series = gdf["area_m2"]
    return pd.DataFrame({
        name: (series >= catalog_min_area[name]) & (series <= catalog_max_area[name])
        for name in catalog.keys()
    })
```

```
def check_soil(gdf, catalog):
  """ if soil data exists, it can also be checked againt custom BGI requirements """
    catalog_soils = {
        name: set(entry["soil_type"])
        for name, entry in catalog.items()
    }
    series = gdf["soil"]
    return pd.DataFrame({
        name: series.isin(layers)
        for name, layers in catalog_layers.items()
    })
```

```
def check_groundwater(gdf, catalog):
  """ if groundwater levels exist, they can also be checked against custom BGI requirements """
    catalog_groundwater = {
        name: set(entry["groundwater_level"])
        for name, entry in catalog.items()
    }
    series = gdf["groundwater"]
    return pd.DataFrame({
        name: series.isin(layers)
        for name, layers in catalog_layers.items()
    })
```

Applying masks (binary checks)

```
##############################################################################
# creating final masks and output shapefile
##############################################################################

valid_mask = pd.Series(True, index=BGI_CATALOG.keys())
area_mask = check_area(gdf, BGI_CATALOG)
lulc_mask = check_install_layer(gdf, BGI_CATALOG)
final_mask = area_mask & lulc_mask

# making the mask go from boolean to a readable result
gdf['suitable_bgis'] = final_mask.apply(
    lambda row: [bgi for bgi, valid, in row.items() if valid],
    axis = 1
)
```

* In the Amsterdam example, we do not provide soil data or groundwater height data. Therefore, the corresponding checks are not applied. This is how you can disregard constraints you are not interested in applying.

Format and save your results

Creates a matrix showing if a BGI type can or cannot be applied at each polygon where:

0 = cannot be applied
1 = can be applied

This allows you to easily filter for all polygons where a specific BGI can be applied in QGIS

```
# add binary matrix of 0/1s for easier visualization in QGIS
bgi_binary_matrix = pd.DataFrame(
    0,
    index = gdf.index,
    columns = list(BGI_CATALOG.keys())
)
for idx, bgis in gdf['suitable_bgis'].items():
    if bgis:
        bgi_binary_matrix.loc[idx, bgis] = 1

output_gdf = pd.concat([gdf, bgi_binary_matrix], axis=1)
```

```
# save file
output_gdf.to_file("testing_standalone.gpkg", layer="feasibility", driver="GPKG")
```

The file name your results will be saved under

The end!

- Email kim.wang@wur.nl for any questions